Secure Three-Tier App: SOLE ATELIER

IMPLEMENTED	TESTED (LOCAL ONLY)	DEPLOYED	PLANNED
Storefront app, tests, docs, runbooks	91 tests, 35 browser checks, SQLite	Nothing. No AWS resource exists	Manual AWS build, then evidence

1 What it does and why

A working sneaker storefront (catalogue, per-size stock, cart, demo checkout) designed for a three-tier AWS build done by hand in the Console.

Why it matters: a real app makes the AWS questions real: how does it reach a private database, who can read the password, what does a healthy target group prove?

2 Architecture and flow

User → IGW → ALB → EC2 FastAPI → RDS MySQL

Public subnets hold the ALB, private subnets the app, isolated subnets the database. A NAT gateway gives the app outbound 443 for Secrets Manager and Session Manager.

Design only. Full diagram in architecture.png.

3 Technologies and roles

FastAPI + Jinja2	pages and JSON API (built)
SQLAlchemy	SQLite locally, MySQL on RDS
VPC + subnets	public / private tiers (plan)
Security groups	tier-to-tier rules (plan)
RDS MySQL 8.4	orders and stock (plan)
Secrets Manager	DB and session secrets (plan)
Session Manager	shell with no port 22 (plan)

4 Implementation steps

- LOCAL App built, tested here (CP8 implemented)
- PLANNED Budget, MFA (CP0)
- PLANNED VPC, subnets, route tables (CP1)
- PLANNED Security groups (CP2), RDS (CP3)
- PLANNED IAM, NAT, EC2 (CP4), ALB (CP5)
- PLANNED Negative tests, teardown (CP6, CP7)

No AWS step has been performed.

5 Important commands

```
pytest # 91 passed (run)
python tools/mutation_check.py # 6/6 caught (run)
node tools/browser-smoke.mjs URL # 35/35 (run)
python -m backend.init_db --admin-user admin # not run: needs RDS
```

6 Key concepts in the design

- A subnet is public because of its route, not its name
- Security groups reference security groups
- Liveness vs readiness health checks
- Atomic `UPDATE ... WHERE stock >= n`
- Idempotency key stops double orders
- Server prices; browser sends ids only

7 Evidence status

VERIFIED Local tests, browser run, health checks, contrast (E-015 to E-020)

NOT VERIFIED MySQL, real concurrency, other browsers

PLANNED All AWS evidence, E-001 to E-014

Limits: AZ-a failure likely means an outage (one NAT, one DB).

8 Security notes

- TESTED CSRF, parameterised SQL, escaped HTML, strict CSP
- PLANNED Private DB, SG chain, no SSH, 2-secret role
- GAP HTTP only, no rate limit, no secret rotation

9 Likely interview questions

Healthy target = working app?

No. It proves the ALB reached /health. The database, checkout and security need their own tests.

Last pair, two buyers?

One conditional UPDATE wins; the other gets a 409 and rolls back. Tested in sequence, not under load.

What if AZ a fails?

Probably an outage: NAT and RDS sit in AZ a. Accepted for cost and stated openly.

Interview explanation (30 seconds)

I built a sneaker storefront where the server, not the browser, decides prices and stock, and I tested that by breaking it on purpose. I designed it for a private database behind security-group chaining on AWS, but I have not built or verified the AWS side yet, so I report it as planned.

Source: this repository's files and recorded local runs only. Nothing here is a claimed deployment or metric.

MG